

Interference-Aware Routing: Evaluation Report

Autonomous Networks Research Group (ANRG)
University of Southern California

1 Experiment Setup

Simulator: ncsim, HEFT scheduler, `csmabianchi` 802.11ax interference model. **RF config:** 5 GHz, 20 MHz channel, $P_{\text{tx}} = 20$ dBm, path-loss exponent $n = 3.0$. **Grid spacing:** 40 m (adjacent links ≈ 8.6 MB/s PHY; diagonals ≈ 56.6 m, lower PHY rate). **Seeds:** Each configuration averaged over 30 seeds (smoothing HEFT non-determinism from Python hash randomization). **Task config:** `compute_cost` = 500, `data_size` = 10 MB per edge.

1.1 Networks

Network	Nodes	Links	Topology
4×4 grid	16	68	Grid + checkerboard diagonals, heterogeneous compute
7×7 grid	49	240	Grid + checkerboard diagonals, heterogeneous compute

1.2 DAGs

DAG	Tasks	Edges	Structure
Small	8	12	Fork-join: 1 source $\rightarrow$ 6 parallel $\rightarrow$ 1 sink
Large (4×4)	30	48	5-stage pipeline with cross-connections
Large (7×7)	60	102	6-stage pipeline with cross-connections

Label	Scheme	Description
W	Widest-Path	Max bottleneck bandwidth; ignores interference
S	Shortest-Path	Min total latency ($\equiv$ min hops at equal latency); ignores interference
GS	Greedy-Start	Static greedy at DAG inject; flows sorted by est. start time
GC	Greedy-Criticality	Static greedy; flows sorted by HEFT upward rank (desc.)
GB	Greedy-Bytes	Static greedy; flows sorted by data size (desc.)
GO	Greedy-Overlap	Static greedy; flows sorted by overlap degree (desc.)
GSD	Greedy-Start Dynamic	Routes computed at <code>transfer_start</code> using actual active-link state

Experiment	W	S	GS	GC	GB	GO	GSD	Best
4×4 , small	94.4	87.4	37.1	37.4	39.0	38.9	39.4	GS
4×4 , large	427.6	326.3	201.3	207.6	207.9	211.4	374.8	GS
7×7 , small	466.5	109.9	248.9	262.0	297.4	224.7	167.4	S
7×7 , large	5130.0	4102.3	3583.4	3617.6	3540.8	3619.1	3446.6	GSD

1.3 Schemes Evaluated

2 Results

2.1 Mean Makespan (seconds, averaged over 30 seeds)

2.2 Improvement over Widest-Path Baseline

2.3 Winners

3 Analysis

3.1 Static Greedy (GS) Dominates on the 4×4 Grid

On the smaller 4×4 grid, all four greedy variants (GS/GC/GB/GO) dramatically outperform the baselines, cutting makespan by 51–61%. GS is consistently the best of the four, though GC is within 1–3%. The margin among greedy variants is small because with only 16 nodes and 40 m spacing, most node pairs are 1–2 hops apart, and there are fewer alternative paths to differentiate the orderings.

GSD underperforms on 4×4 large (374.8 s vs GS 201.3 s). This is the most striking result. GSD’s per-flow greedy optimization picks the best path for *each individual transfer* at start time, but cannot coordinate across flows. On a small, dense grid where many transfers share the same few links, the static planner’s global throughput objective produces better overall link utilization. GSD ends up making locally optimal but globally suboptimal choices that cascade through the multi-stage pipeline.

3.2 Shortest-Path Wins on 7×7 with a Small DAG

With only 8 tasks on 49 nodes, HEFT spreads tasks across the grid, creating long-distance transfers. Shortest-Path’s min-hop paths keep transfers on 1–2 hop routes where possible, avoiding the multi-hop relay chains that Widest-Path selects (which traverse more links and trigger more interference).

Experiment	S	GS	GC	GB	GO	GSD
4×4 , small	-7%	- 61%	-60%	-59%	-59%	-58%
4×4 , large	-24%	- 53%	-51%	-51%	-51%	-12%
7×7 , small	- 76%	-47%	-44%	-36%	-52%	-64%
7×7 , large	-20%	-30%	-29%	-31%	-29%	- 33%

Experiment	Winner	Runner-up	Gap
4×4 , small	GS (37.1 s)	GC (37.4 s)	0.6%
4×4 , large	GS (201.3 s)	GC (207.6 s)	3.0%
7×7 , small	S (109.9 s)	GSD (167.4 s)	34.4%
7×7 , large	GSD (3446.6 s)	GB (3540.8 s)	2.7%

The greedy variants (GS = 249 s, GO = 225 s, GSD = 167 s) all lose to S (110 s) here. The static greedy planners suffer because their HEFT timing estimates are inaccurate for such a sparse DAG on a large grid—the concurrency windows they compute don’t match reality. GSD does better (167 s) because it sees actual state, but still can’t beat S’s simple min-hop strategy when there are only 12 transfers and 240 links to spread across.

3.3 GSD Wins on 7×7 with a Large DAG

The 60-task pipeline on the 7×7 grid is the most complex scenario, with 102 edges creating sustained concurrent traffic across the network. Here GSD (3447 s) edges out GB (3541 s) by 2.7%. The real-time state information becomes valuable: with many active transfers, GSD can route each new flow around current hot spots.

The static greedy variants (GS = 3583 s, GC = 3618 s, GB = 3541 s, GO = 3619 s) are all close, with GB slightly ahead—suggesting that prioritizing large transfers in the greedy ordering matters less when transfer sizes are uniform (here all 10 MB, so GB degenerates to an arbitrary tie-break that happens to produce slightly better routes for this topology).

3.4 Greedy Ordering Matters Less Than Expected

Across all four experiments, the four greedy orderings produce makespans within 2–12% of each other. The benchmark uses uniform `data_size` = 10 MB, which neutralizes GB’s advantage, and equal `compute_cost` = 500, which reduces the dynamic range of criticality. GO’s overlap-degree sorting helps on 7×7 small (225 s vs GS 249 s, -10%) where temporal overlap estimation is more predictive.

In workloads with heterogeneous transfer sizes and compute costs, we expect the orderings to diverge more.

3.5 Widest-Path Is Consistently the Worst

W never wins and is 2–14× worse than the best scheme. Its paths maximize raw bandwidth but ignore interference, often selecting long relay chains that traverse many links and trigger severe contention. In wireless mesh networks under `csma_bianchi`, path length and interference matter far more than nominal link bandwidth.

4 Regime Map

Scenario	Best approach	Why
Small grid, any DAG	GS	Few paths, static planner coordinates well
Large grid, small DAG	S	Few transfers, min-hop avoids unnecessary interference
Large grid, large DAG	GSD	Many concurrent flows, real-time state enables adaptation
Uniform transfer sizes	GS/GC/GB/GO	Ordering tie-breaks matter little

5 Implications for Deferral

The results suggest deferral would be most valuable in exactly the regime where GSD already wins (large grid, large DAG):

- **On 4×4 grids**, the network is too small for deferral to help—all paths are short and there are limited alternatives. The bottleneck is coordinating routes, which static GS handles well.
- **On 7×7 with small DAGs**, transfers are infrequent enough that contention is mild. Deferral would add latency without benefit.
- **On 7×7 with large DAGs**, there are many concurrent transfers competing for airtime. A deferred transfer that waits 1–2 transfer-completions might find a much less congested path. The risk is stalling critical-path tasks. A conservative deferral policy (defer only non-critical transfers when current best-path bandwidth is below, say, 30% of the uncongested baseline) could improve on GSD’s 3447s without the starvation risks of aggressive deferral.

6 Reproducing These Results

```
cd ncsim/  
python run_routing_eval.py  
# Output: /tmp/ncsim_routing_eval/eval_results.json  
# Runtime: ~45 minutes
```